

DESIGN DOCUMENT

(Specialization Project - C1 Evaluation)

Sentari Agent-OS

*(An Operating-System-Inspired Kernel for Scheduling, Isolation,
and Deadlock Detection in Multi-Agent AI Systems)*

GitHub Repository: <https://github.com/Vishalsym/Sentari-AgentOS>

Submitted by

Vishal Sharma

Roll No: 25225039

Under the guidance of

Dr. Ramesh Chandra Poonia

Dr. Shilpa Shrivastava



CHRIST

(DEEMED TO BE UNIVERSITY)

BANGALORE | DELHI NCR | PUNE

School of Sciences

2026-2027

1. System Architecture

Sentari Agent-OS is structured as four layers: an Agent Layer (the LLM-driven agents being orchestrated), a Syscall Layer (the sole mediated entry point into the kernel), the Sentari Agent-OS Kernel itself (Scheduler, Memory Manager, Deadlock Detector, Preemption Manager), and a Resource Layer (LLM API, external tools, and the shared knowledge base). Every arrow in the diagram below represents a mediated, logged call — no layer is permitted to skip the one below it.

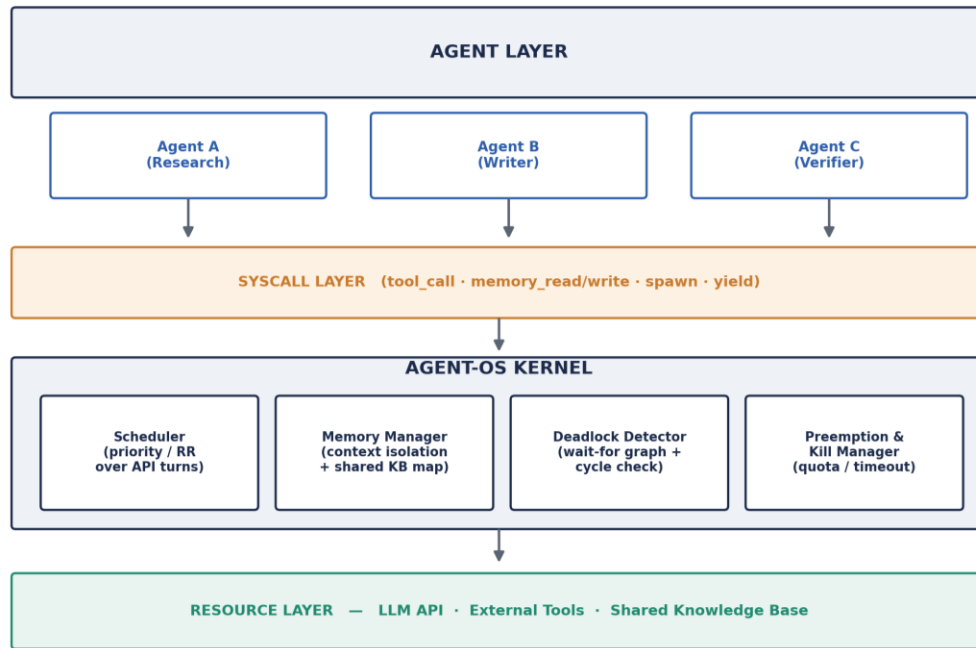


Figure 1. Sentari Agent-OS layered system architecture.

2. Module Design

2.1 Scheduler Module

Maintains the ready queue and dispatches agents for API/tool turns. Supports two interchangeable policies: round-robin (equal time-slices) and priority-preemptive (higher-priority agents dispatched first, with aging to prevent starvation of low-priority agents). Directly extends classical CPU-scheduling logic, substituting “LLM API call slot” for “CPU time slice.”

2.2 Syscall Layer Module

Exposes a small, fixed set of syscalls: `tool_call`, `memory_read`, `memory_write`, `spawn`, and `yield`. Each syscall is validated against the calling agent's PCB (permissions, remaining quota) before being forwarded to the Resource Layer. Every syscall, whether accepted or denied, is written to the audit log with a timestamp, agent ID, and result.

2.3 Memory Manager Module

Allocates each agent a private, isolated context store (analogous to a process address space). Access to the shared knowledge base is only possible through explicit `memory_read` / `memory_write` syscalls, which the Memory

Manager resolves through a mapping table — conceptually a page table — from (agent_id, logical_key) to a physical record in the shared store.

2.4 Deadlock Detector Module

Maintains a wait-for graph: a directed edge Agent(X) → Agent(Y) is added whenever X blocks pending a resource currently held by Y. On every new edge, the module runs a cycle-detection pass (depth-first search maintaining a recursion stack). If a cycle is found, it is reported to the Preemption Manager with the full cycle path.

2.5 Preemption & Kill Manager Module

Acts on three triggers: (a) an agent's quota or time-slice is exhausted, (b) an agent has been blocked past a configurable timeout, or (c) the Deadlock Detector reports a cycle. It selects a victim (lowest priority / most quota already consumed), transitions its PCB to KILLED or back to READY, and releases all resources it held.

2.6 Logging & Observability Module

Persists PCB snapshots and the syscall audit log to SQLite, and optionally serves a live view of agent state, quota usage, and the wait-for graph to the dashboard described in Section 3.

3. UI Design — Monitoring Dashboard

The dashboard is a read-only observability surface for developers; it does not issue commands into the kernel. It is organized into three panels: an active-agent / PCB panel on the left, a wait-for graph and quota-usage panel in the center, and a live syscall-log stream on the right.

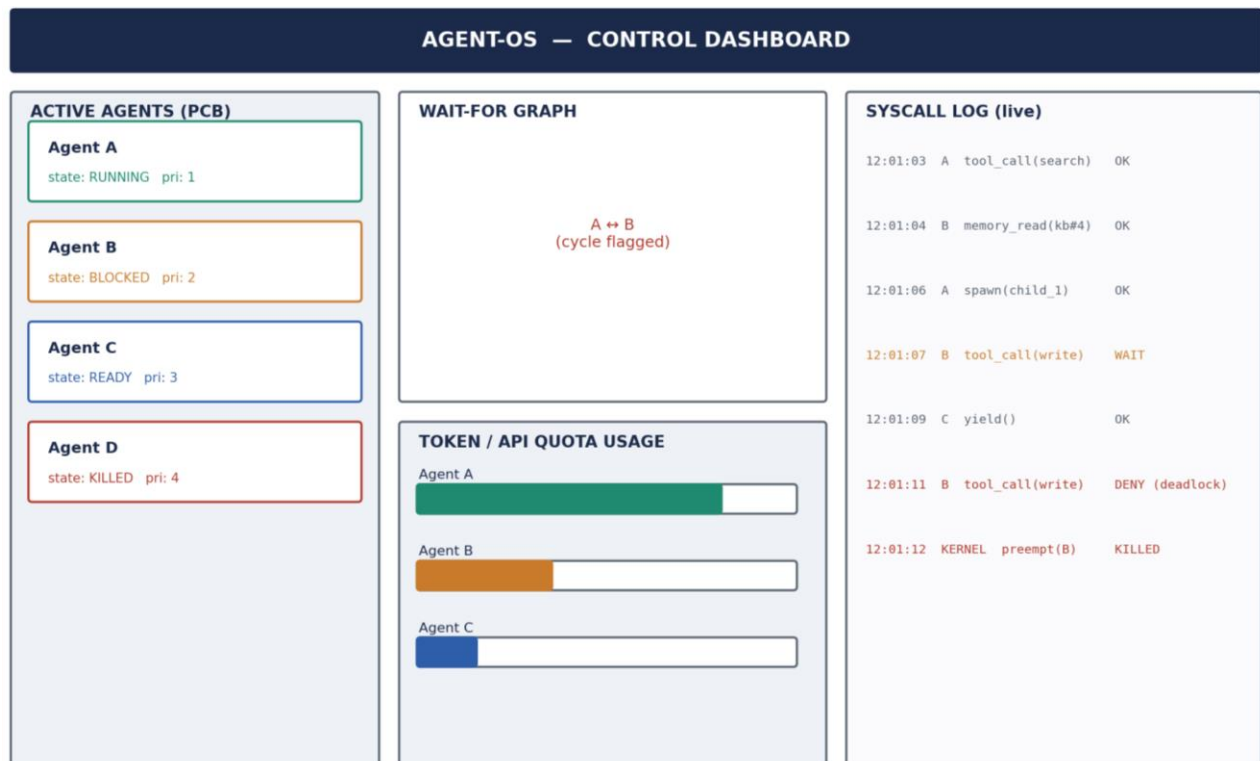


Figure 2. Control-dashboard wireframe (low-fidelity mockup).

3.1 Panel Descriptions

- Active Agents (PCB): lists every registered agent with its current state (colour-coded) and priority.
- Wait-for Graph: renders the live resource-allocation graph; a detected cycle is highlighted in red with the cycle path labelled.
- Token / API Quota Usage: horizontal bars showing quota consumed vs. remaining per agent, updated per syscall.
- Syscall Log: a scrolling, timestamped stream of every syscall with its outcome (OK / WAIT / DENY / KILLED).

4. Database Design

Sentari Agent-OS persists state to a lightweight relational store (SQLite for the reference implementation). The schema below covers the four tables needed to reconstruct any point-in-time system state from the log.

4.1 Table: agents (PCB store)

Column	Type	Description
agent_id	TEXT (PK)	Unique identifier for the agent
parent_id	TEXT (FK → agents.agent_id)	Spawning parent agent, if any
state	TEXT	NEW READY RUNNING BLOCKED TERMINATED KILLED
priority	INTEGER	Scheduling priority (lower = higher priority)
quota_total	INTEGER	Total token/API-call budget assigned
quota_used	INTEGER	Quota consumed so far
created_at	DATETIME	PCB creation timestamp
updated_at	DATETIME	Last state-transition timestamp

4.2 Table: syscall_log

Column	Type	Description
log_id	INTEGER (PK, autoincrement)	Unique log entry ID
agent_id	TEXT (FK → agents.agent_id)	Agent that issued the syscall
syscall_type	TEXT	tool_call memory_read memory_write spawn yield
arguments	TEXT (JSON)	Serialized syscall arguments
result	TEXT	OK WAIT DENY ERROR
timestamp	DATETIME	Time the syscall was issued

4.3 Table: resource_allocation

Column	Type	Description
allocation_id	INTEGER (PK, autoincrement)	Unique allocation record ID
resource_key	TEXT	Logical identifier of the held/awaited resource
holder_agent_id	TEXT (FK → agents.agent_id)	Agent currently holding the resource (nullable)
waiter_agent_id	TEXT (FK → agents.agent_id)	Agent waiting on the resource (nullable)
created_at	DATETIME	When this hold/wait edge was recorded

4.4 Table: knowledge_base

Column	Type	Description
kb_key	TEXT (PK)	Logical key for a shared knowledge item
value	TEXT	Stored content (serialized)
last_writer_agent_id	TEXT (FK → agents.agent_id)	Agent that last wrote this key
updated_at	DATETIME	Last write timestamp

5. Agent Lifecycle (State Design)

Every agent's PCB moves through a well-defined set of states. This state machine underpins both the Scheduler and the Preemption Manager: a transition into BLOCKED registers a wait-for edge; a transition into KILLED releases every resource the agent held.

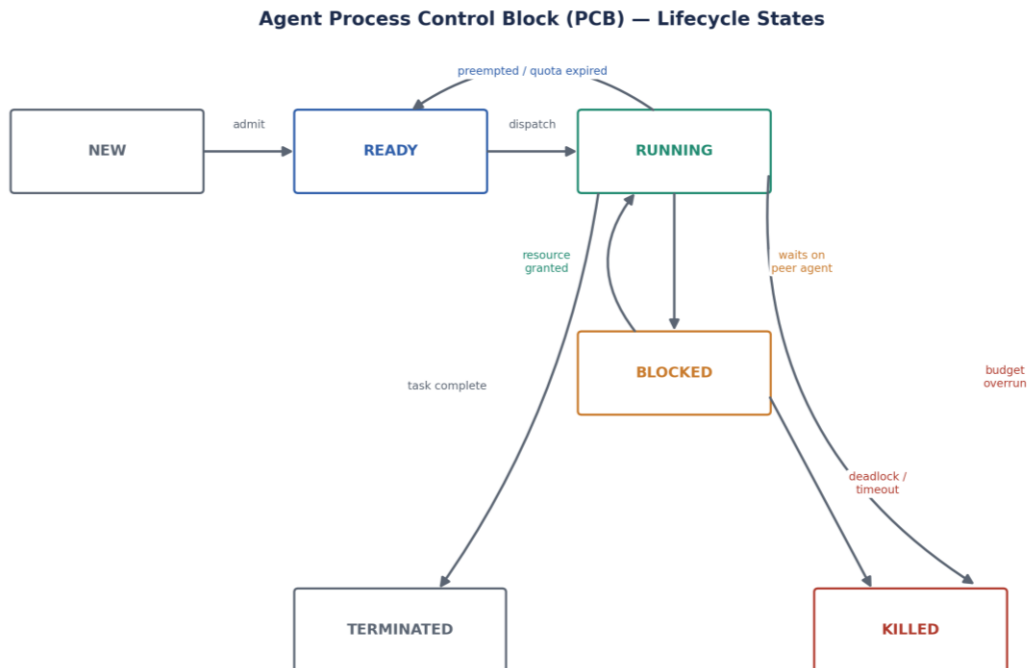


Figure 3. Agent PCB lifecycle state diagram.

6. Algorithm Design

6.1 Resource-Aware Scheduling Algorithm

Every syscall first passes through the quota check below before it is ever enqueued for dispatch:

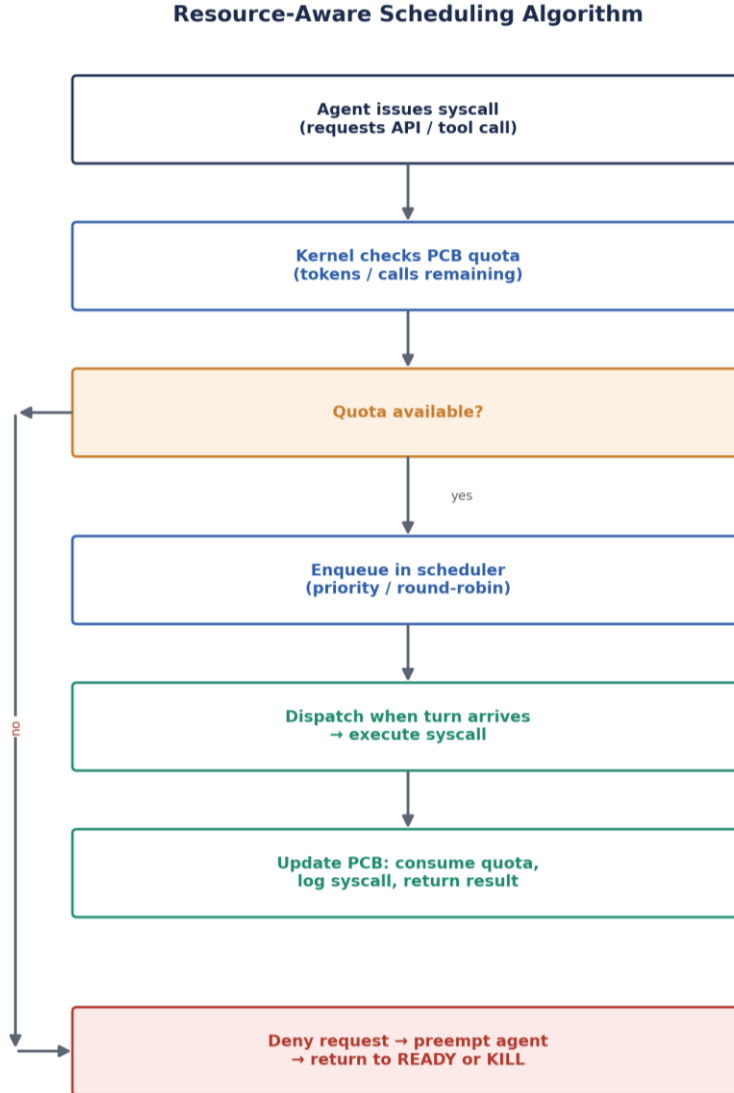


Figure 4. Scheduling / admission-control flowchart.

```

function on_syscall(agent, request):
    if agent.quota_used >= agent.quota_total:
        preempt(agent); return DENY
    enqueue(scheduler_queue, agent, request)
    wait_for_turn(agent)
    result = execute(request)          # dispatched turn
    agent.quota_used += cost(request)
    log(agent, request, result)
    return result
  
```

6.2 Deadlock Detection Algorithm (Cycle Detection over Wait-For Graph)

When Agent A blocks waiting on a resource held by Agent B, an edge $A \rightarrow B$ is added to the wait-for graph and a depth-first cycle check runs immediately:

Cycle detected → DEADLOCK

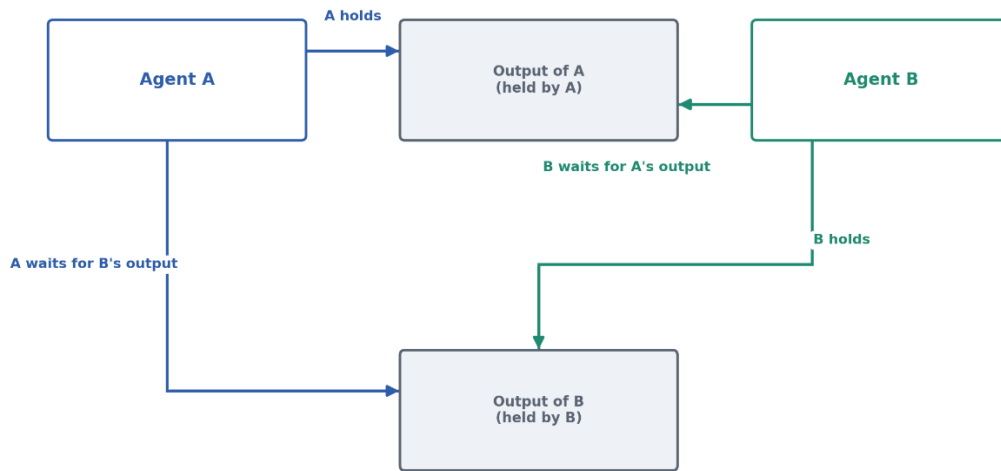


Figure 5. Example wait-for cycle between two agents.

```

function add_wait_edge(waiter, holder):
    graph.add_edge(waiter, holder)
    if has_cycle(graph, start=waiter):
        cycle = extract_cycle(graph, waiter)
        victim = min(cycle, key=lambda a: a.priority)
        preempt_or_kill(victim)
        graph.remove_edges_for(victim)

function has_cycle(graph, start):
    visited, stack = set(), set()
    return dfs(start, visited, stack)

function dfs(node, visited, stack):
    visited.add(node); stack.add(node)
    for neighbor in graph.edges(node):
        if neighbor in stack: return True          # back-edge = cycle
        if neighbor not in visited and dfs(neighbor, visited, stack): return True
    stack.remove(node); return False
  
```

Complexity: each cycle check is $O(V + E)$ over the current wait-for graph; since checks run incrementally on each new edge rather than by periodic full-graph sweeps, detection is near-immediate (bounded by graph size, which is bounded by the number of concurrently blocked agents).

7. Traceability Note

Sections 2, 4, 5, and 6 of this design document map directly onto Functional Requirements FR-1 through FR-17 defined in the SRS: Section 2.1–2.6 realize FR-1–FR-3 and FR-15–FR-17; Section 6.1 realizes FR-4–FR-9; Section 6.2 realizes FR-12–FR-14; Section 4 realizes the persistence needed for FR-9 and FR-17.